Embedded System Concepts

A computing system fundamentally consists of a **Processor**, **Memory**, and **I/O (Input/Output)** units. It can be divided into two main types: General Purpose and Specific Purpose.

Embedded System Constraints

When designing an embedded system, several challenges or constraints must be considered:

- Power consumption
- Size
- Speed / Performance time
- Cost

Embedded System Methods

There are two primary methods for building embedded systems:

1. **System Board (SB):** Assembling separate components on a single board.
2. **System on Chip (SoC):** Integrating the entire system into a single microchip.

Comparison Aspect	System Board (SB)	System on Chip (SoC)
Cost	High (↑)	Low (↓) ✔️
Size	Large (↑)	Small (↓) ✔️
Performance (Speed)	The Same	The Same
Power Consumption	High (↑)	Low (↓) ✔️
Configurability	Yes (✔️)	No (❌)

Note: Generally, SoC is better than SB in most factors (Cost, Size, Power). However, in the initial design stages, SB is preferred because it offers configurability and the ability to test or modify components before final production.

Processor Types & Systems

- **MPU (Microprocessor Unit):** A modern processor made from tiny transistors, containing an ALU, a Control Unit, and Registers.
- **CPU (Central Processing Unit):** The primary processor in the system. There are secondary processors that assist the CPU (like GPU and DSP) to reduce its load.
- **MCU (Microcontroller Unit):** A small computer that integrates the three parts of any computing system (Processor, Memory, I/O) into a single unit.

- **SoC (System on Chip):** Acts as a high-performance MCU, integrating the primary processor with secondary processors (GPU, DSP) on one chip.

Bare Metal Software & Structure

- **Software Flow:** APP → Drivers → HW (Hardware).
- **Kit:** An SoC or MCU combined with simple additions like LEDs.
- **ECU (Electronic Control Unit):** A Kit integrated with sensors and actuators. It is the foundational building block for large systems like automobiles.

Instruction Life Cycle

Every instruction goes through three sequential stages:

1. **Fetch:** The instruction is brought from the memory (ROM) into the processor. The Program Counter (PC) points to the location of the next instruction. Once fetched, it is stored in an internal register called the Instruction Register (IR).
2. **Decode:** Happens inside the Control Unit to understand the instruction. This stage depends on the processor's Instruction Set and Instruction Format to decode the operation and send the suitable signals to the ALU.
3. **Execute:** The ALU receives the signal and the operands, performs the required operation, and outputs the result to be stored temporarily in a general-purpose register.

Instruction Decoder (ID) Type	Characteristics & Usage
Hard-wired	Built with logic gates. Used with simple systems and RISC architecture.
Memory Mapped	Relies on searching in memory. Used with CISC architecture.

Instruction Set Architecture (ISA)

There are two main architectures: **RISC** (Reduced Instruction Set Computer) and **CISC** (Complex Instruction Set Computer).

Embedded Systems Challenges	RISC	CISC
Size	ALU is small (↓), ID is large (↑)	ALU is large (↑), ID is small (↓)
Cost	Software is High (↑), Hardware is Low (↓)	Software is Low (↓), Hardware is High (↑)

Embedded Systems Challenges	RISC	CISC
Performance	Same	Same
Power Consumption	ALU is low (↓), ID is high (↑)	ALU is high (↑), ID is low (↓)
Decoder	Hard-wired	Memory-mapped
Compiler	Strong/Complex	Simple

Register Files

- **General Purpose Registers:** Used to store data temporarily.
- **Specific Purpose Registers:**
    - **PC (Program Counter):** Points to the next instruction.
    - **IR (Instruction Register):** Stores the instruction that is currently being decoded.
    - **SP (Stack Pointer):** Points to the last filled or next available location in the memory.
    - **Sign Flag:** Indicates if the result is positive (+ve) or negative (-ve).
    - **Zero Flag:** Indicates if the result equals zero.

Memory Basics

Memory consists of locations, and each location holds a number of bits. The basic building block of memory is the **Flip-Flop**.

- The processor connects to the memory via 3 main buses: **Address Bus**, **Data Bus**, and **Control Bus**.
- **Read/Write Operations:** To read, the processor sends the address on the Address Bus and a read signal on the Control Bus; then the memory sends the data via the Data Bus. Writing data to memory takes more time than reading from it.
- The number of Address Bus lines determines the maximum size of the memory (Size = 2^n, where n = number of lines).
- **Characteristics for any memory:** 1. Capacity, 2. Access time, 3. Organization.

Volatile Memory (RAM)

Comparison	Dynamic RAM (DRAM)	Static RAM (SRAM)
Mechanism	Uses Capacitors (1 = Charged, 0 = Discharged).	Uses Transistors (at least 6 transistors per bit).
Problem	Capacitors leak charge over	High cost and low density.

Comparison	Dynamic RAM (DRAM)	Static RAM (SRAM)
	time.	
Solution	Requires a "refresh circuit" to keep data alive.	Stable internal loop, doesn't need a refresh circuit.
Speed & Cost	Slow, but high density and low cost.	Very fast compared to DRAM.

Non-Volatile Memory (ROM)

The CPU can only read from it. To write on ROM, an external device called a "Burner", "Flasher", or "Flash driver" is needed. This device applies a high voltage to a "Control gate" to break through an insulating layer (SiO_2) and force electrons into a "Floating gate".

- **Mask ROM:** Programmed only once at the factory. Very cheap and used for mass production.
- **PROM (Programmable ROM):** One-Time Programmable (OTP) by the user. It depends on internal fuses; once a fuse is burned to write code, it cannot be changed.
- **EPROM (Erasable PROM):** Can be erased using ultraviolet light through a small window. Requires erasing the entire memory at once and is sensitive to radiation and noise.

Hybrid Memory

Hybrid memory mixes the read/write capabilities of RAM with the non-volatile nature of ROM.

- **EEPROM (Electrically Erasable PROM):** Can be electrically erased and written. It has high endurance (up to 100,000 cycles) and features **Byte Access** (you can erase/modify a single byte). Due to its high cost, it is used for small, critical data like passwords.
- **Flash Memory:** Also electrically erasable but much faster than EEPROM for erasing large blocks because it uses **Sector Access** (erasing sector by sector). It has lower endurance (approx. 10,000 cycles) but is much cheaper, allowing for high capacity. Primarily used for storing program code.
- **NVRAM (Non-Volatile RAM):** A combination of fast SRAM, a small battery, and EEPROM. It operates at SRAM speeds. When main power is lost, the battery provides enough temporary power to save critical data from the SRAM into the EEPROM. When power returns, data is restored.